디자인 패턴

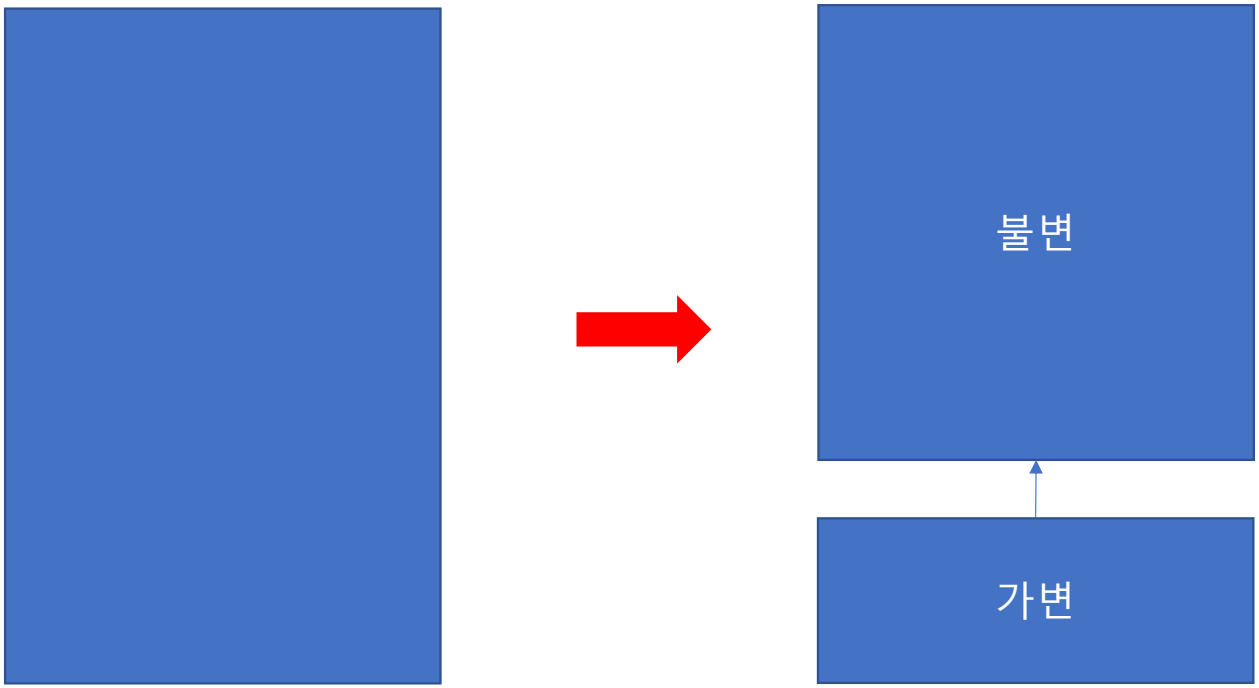
2026. 3. 31 w3
미림 마이스터고

남궁성
seong.namkung@gmail.com

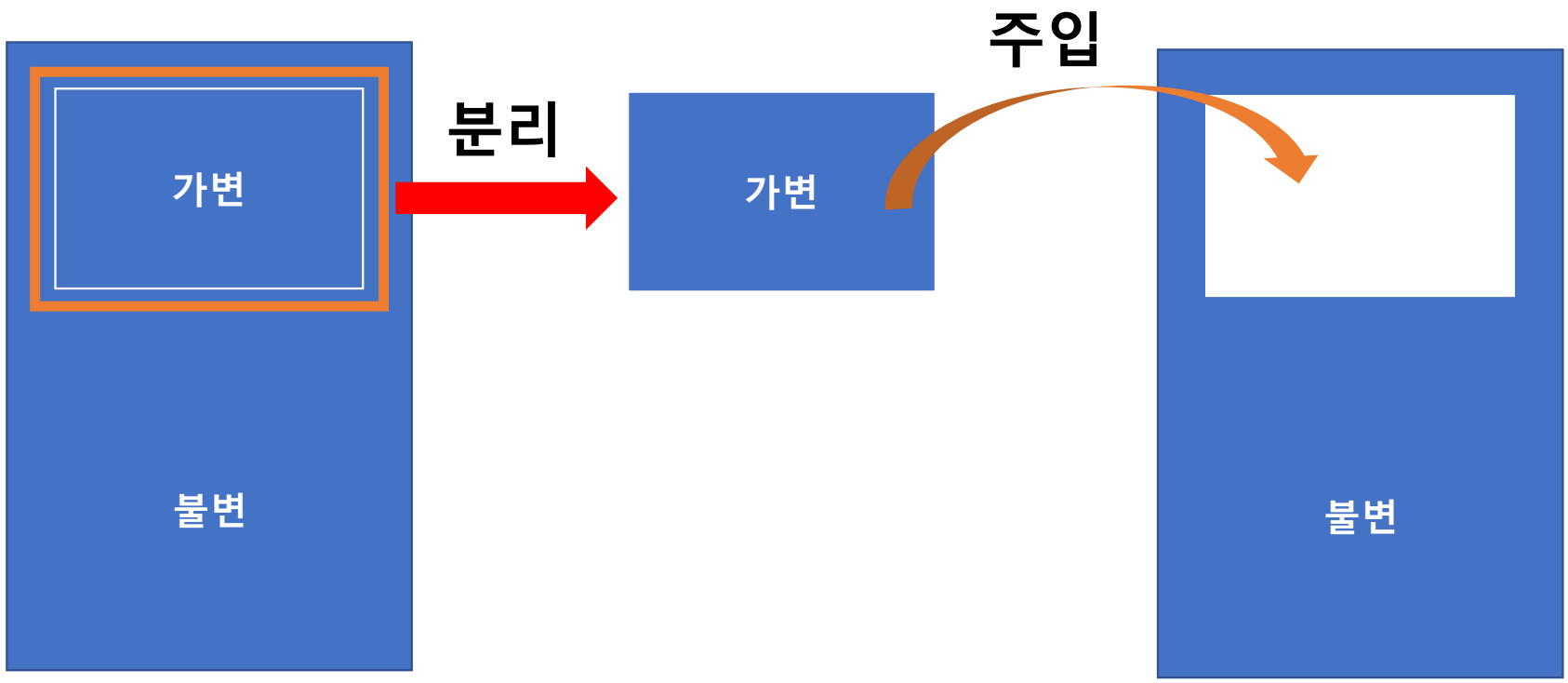
분리

1. 관심사(SRP)
2. 가변과 불변 (OCP)
3. 중복코드

변하는 것의 분리#1 – Template Method



변하는 것의 분리#2 – Strategy



7. Bridge – 상속 & 주입을 동시에

```java
public class Display {  no usages
    private DisplayImpl impl;  4 usages
    public Display(DisplayImpl impl) {
        this.impl = impl;
    }
    public void open() {  no usages
        impl.rawOpen();
    }
    public void print() {  1 usage
        impl.rawPrint();
    }
    public void close() {  1 usage
        impl.rawClose();
    }
    public final void display() {  no usage
        print();
        close();
    }
}
```

```java
public abstract class DisplayImpl {
    public abstract void rawOpen();
    public abstract void rawPrint();
    public abstract void rawClose();
}
```

주입
(포함)

상속

```java
public class CountDisplay extends Display {
    public CountDisplay(DisplayImpl impl) {
        super(impl);
    }
    public void multiDisplay(int times) {  no
        open();
        for (int i = 0; i<times;i++) {
            print();
        }
        close();
    }
}
```

5. Template Method – 템플릿(공통)을 상속받아 완성

```
class Marine {      // 보병
    int x, y;       // 현재 위치
    void move(int x, int y) { /* 지정된 위치로 이동 */}
    void stop()             { /* 현재 위치에 정지 */  }
    void stimPack()         { /* 스팀팩을 사용한다.*/ }
}

class Tank {        // 탱크
    int x, y;       // 현재 위치
    void move(int x, int y) { /* 지정된 위치로 이동 */   }
    void stop()             { /* 현재 위치에 정지 */     }
    void changeMode()       { /* 공격모드를 변환한다.*/  }
}

class Dropship {    // 수송선
    int x, y;       // 현재 위치
    void move(int x, int y) { /* 지정된 위치로 이동 */   }
    void stop()             { /* 현재 위치에 정지 */     }
    void load()             { /* 선택된 대상을 태운다.*/ }
    void unload()           { /* 선택된 대상을 내린다.*/ }
}
```

```
abstract class Unit {
    int x, y;
    abstract void move(int x, int y);
    void stop() { /* 현재 위치에 정지 */ }
}

class Marine extends Unit {   // 보병
    void move(int x, int y) { /* 지정된 위치로 이동 */    }
    void stimPack()         { /* 스팀팩을 사용한다.*/     }
}

class Tank extends Unit {     // 탱크
    void move(int x, int y) { /* 지정된 위치로 이동 */    }
    void changeMode()       { /* 공격모드를 변환한다. */  }
}

class Dropship extends Unit { // 수송선
    void move(int x, int y) { /* 지정된 위치로 이동 */    }
    void load()             { /* 선택된 대상을 태운다.*/  }
    void unload()           { /* 선택된 대상을 내린다.*/  }
}
```

5. Template Method – 템플릿(공통)을 상속받아 완성

```
abstract class Player {
    boolean pause;          // 일시정지 상태를 저장하기 위한 변수
    int currentPos;         // 현재 Play되고 있는 위치를 저장하기 위한 변수

    Player() {              // 추상 클래스도 생성자가 있어야 한다.
        pause = false;
        currentPos = 0;
    }
    /** 지정된 위치(pos)에서 재생을 시작하는 기능이 수행하도록 작성되어야 한다. */
    abstract void play(int pos);    // 추상 메서드
    /** 재생을 즉시 멈추는 기능을 수행하도록 작성되어야 한다. */
    abstract void stop();           // 추상 메서드

    void play() {
        play(currentPos);           // 추상 메서드를 사용할 수 있다.
    }

    void pause() {
        if(pause) { // pause가 true일 때(정지상태)에서 pause가 호출되면,
            pause = false;          // pause의 상태를 false로 바꾸고,
            play(currentPos);       // 현재의 위치에서 play를 한다.
        } else {    // pause가 false일 때(play상태)에서 pause가 호출되면,
            pause = true;           // pause의 상태를 true로 바꾸고
            stop();                 // play를 멈춘다.
        }
    }
}
```

```
class CDPlayer extends Player {
    void play(int currentPos) {
        /* 조상의 추상 메서드를 구현. 내용 생략 */
    }

    void stop() {
        /* 조상의 추상 메서드를 구현. 내용 생략 */
    }

    // CDPlayer클래스에 추가로 정의된 멤버
    int currentTrack; // 현재 재생 중인 트랙

    void nextTrack() {
        currentTrack++;
            ...
    }

    void preTrack() {
        if(currentTrack > 1) {
            currentTrack--;
        }
            ...
    }
}
```

6. Strategy – 주입 받아서 완성

```java
public class MyTreeMapTest {
    public static void main(String[] args) {
        // 매번 다른 Comparator를 주입 가능
        MyTreeMap map = new MyTreeMap(Comparator.naturalOrder());
        map.put("1", "aaa");
    }
}
```

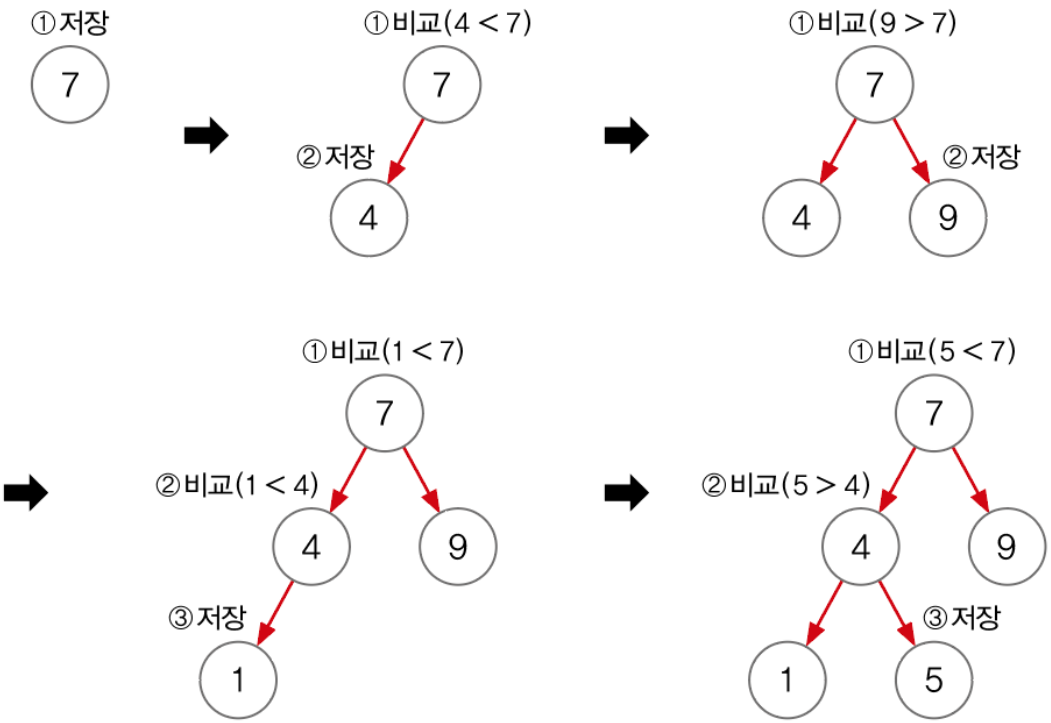
```java
public class MyTreeMap implements Map {  no usages
    private final Comparator comparator;  1 usage

    Change signature
    public MyTreeMap(Comparator comparator) {  no usages
        this.comparator = comparator;
    }
```

주입

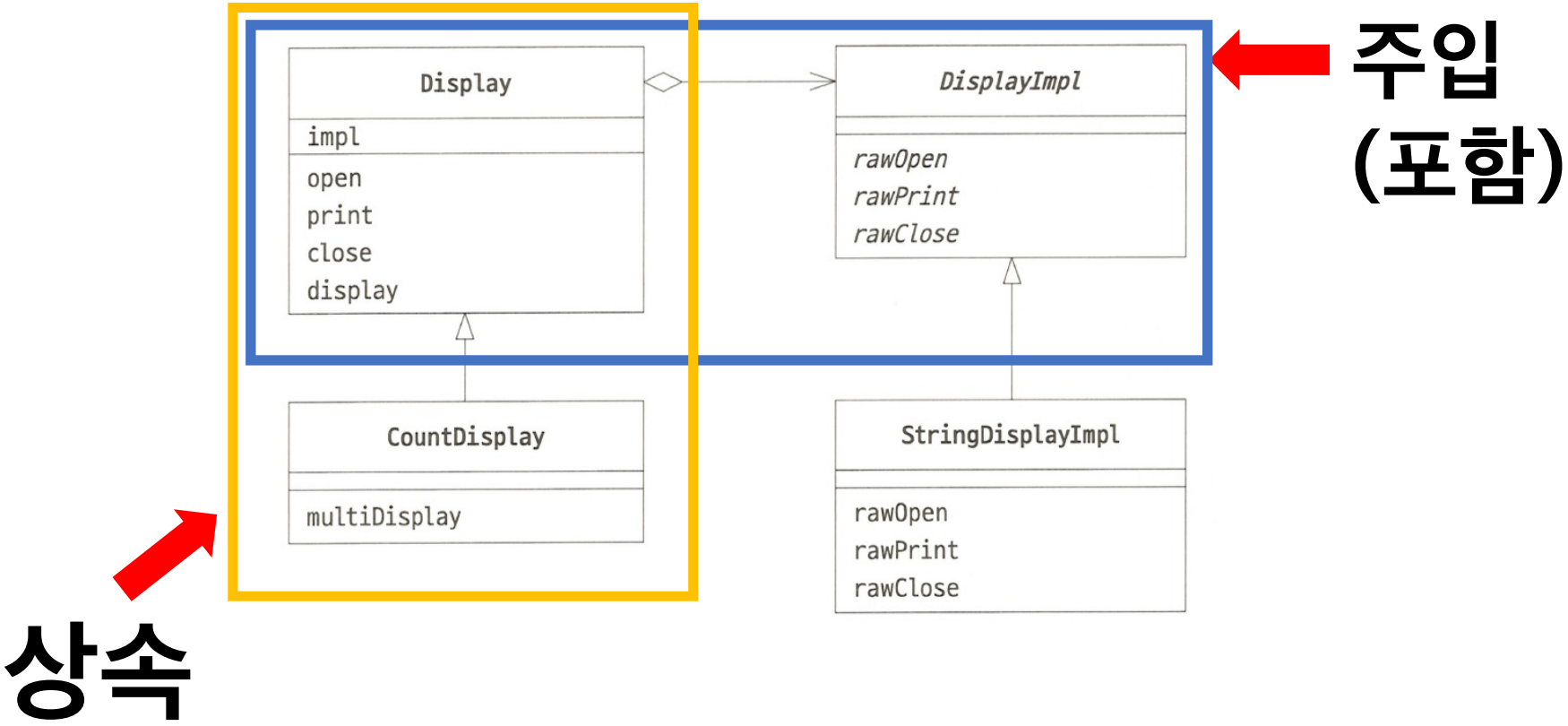
6. Strategy – 주입 받아서 완성

TreeSet – 새로운 요소를 저장하는 과정

7, 4, 9, 1, 5의 순서로 데이터를 저장 – 반복적인 비교로 저장할 위치를 찾음



7. Bridge – 상속 & 주입을 동시에



7. Bridge – 상속 & 주입을 동시에

```java
public class Display {  no usages
    private DisplayImpl impl;  4 usages
    public Display(DisplayImpl impl) {
        this.impl = impl;
    }
    public void open() {  no usages
        impl.rawOpen();
    }
    public void print() {  1 usage
        impl.rawPrint();
    }
    public void close() {  1 usage
        impl.rawClose();
    }
    public final void display() {  no usage
        print();
        close();
    }
}
```

```java
public abstract class DisplayImpl {
    public abstract void rawOpen();
    public abstract void rawPrint();
    public abstract void rawClose();
}
```

주입
(포함)

상속

```java
public class CountDisplay extends Display {
    public CountDisplay(DisplayImpl impl) {
        super(impl);
    }
    public void multiDisplay(int times) {  no
        open();
        for (int i = 0; i<times;i++) {
            print();
        }
        close();
    }
}
```